

Training a Reinforcement Learning Agent for Pokémon Nuzlocke Battles

DSPRO2 Final Presentation

Tristan Muri Marvin Hueber
Viacheslav Godovskii Alex Liubov

Hochschule Luzern – DSPRO2 FS26

May 2026

Agenda

- ➊ **The Challenge** – Nuzlocke rules, Pokémon battles, milestones
- ➋ **Technical Setup** – Architecture, tech stack, training flow
- ➌ **Model & Training** – Observation space, architecture, reward design
- ➍ **Development Journey** – Critical bugs and breakthroughs
- ➎ **Results** – Milestone 1 & 2 outcomes
- ➏ **Live Demo**
- ➐ **Conclusions & Next Steps**

What is a Pokémon Nuzlocke?

A **self-imposed difficulty rule set** that transforms casual Pokémon battles into a problem requiring long-horizon planning, risk assessment, and resource management:

Nuzlocke Rules

- **First-catch:** Only the first Pokémon encountered per location may be captured.
- **Permadeath:** Fainted Pokémon are permanently lost.
- **Run failure:** Losing a battle ends the entire run.

Why this is interesting for RL:

- Irreversible consequences create a *sparse, high-stakes* reward landscape.
- Limited resources require strategic allocation across a sequence of battles.
- Partial observability (unknown opponent teams, hidden abilities).

Pokémon Battles in 30 Seconds

Turn-based 6v6 singles:

- Each player brings up to 6 Pokémon; 1 active at a time.
- Each turn: choose *attack* or *switch* (simultaneous, hidden).
- Execution order: move priority $>$ speed stat.
- Objective: faint all opponent Pokémon.

Type matchups stack multiplicatively:

Modifier	Multiplier
STAB (same-type bonus)	$\times 1.5$
Super effective	$\times 2.0$
Resisted	$\times 0.5$
Immune	$\times 0.0$

Why it's hard for RL

- Large state space (species, types, moves, items, abilities, boosts, weather, ...)
- Stochastic damage rolls
- Simultaneous hidden actions
- Partial observability of opponent team
- Long episodes (~ 25 turns)

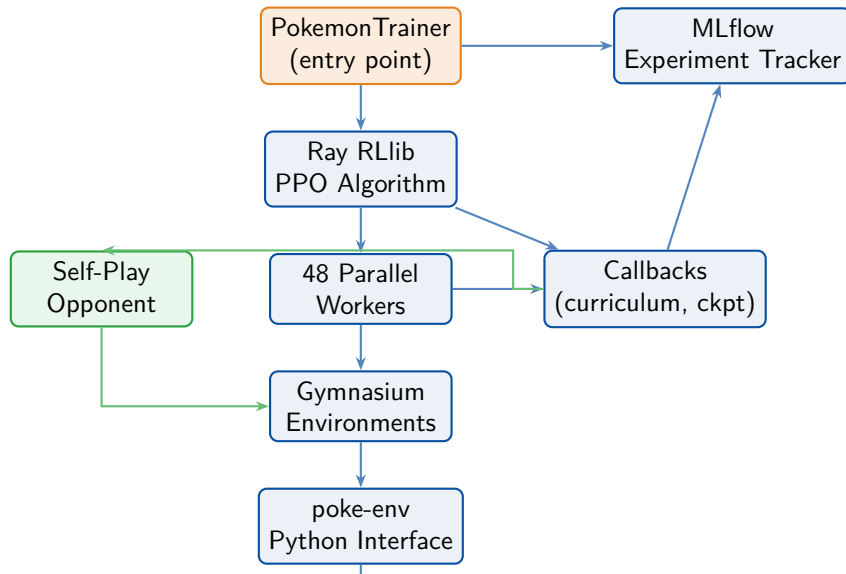
Milestone Roadmap

#	Milestone	Goal	Status
1	Proof of Concept	Fixed team vs. random opponents. Tests whether the model can learn to use a known team.	Solved
2	Random Battles	Randomly generated teams on both sides. Tests generalisation under high entropy.	In progress
3	Team Building	Learn to compose optimal teams given opponent information.	Future
4	Gauntlet Master	Full Nuzlocke gauntlet completion (706 trainers).	Future

Important

The project will **not** be completed within the scope of this module. Milestone 1 is essentially solved; Milestone 2 remains an open challenge. The project will continue beyond the course.

System Architecture

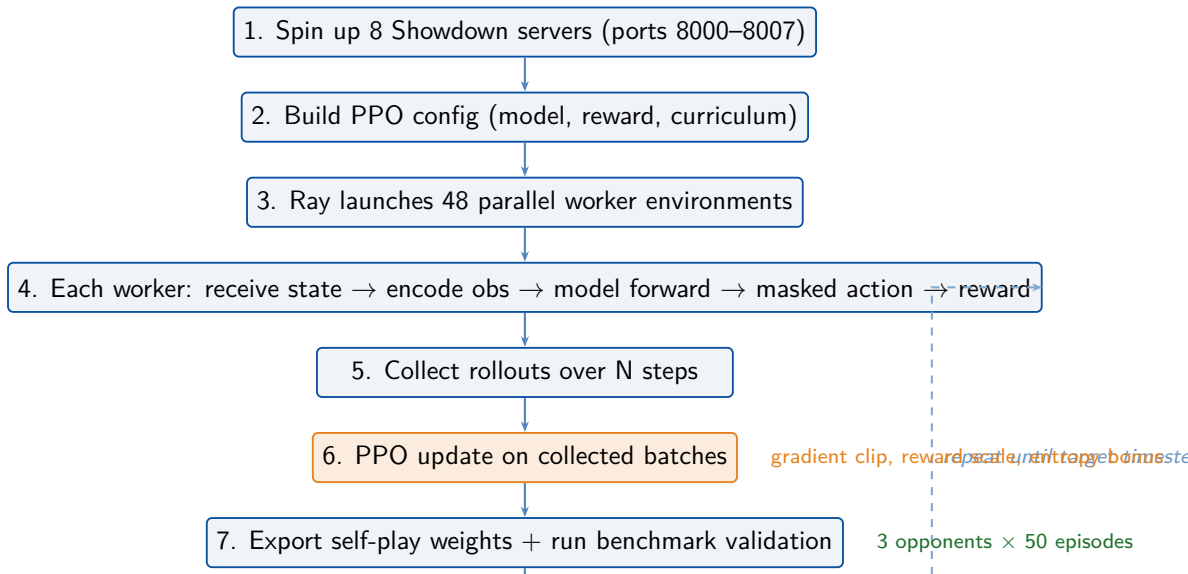


Tech Stack & Versions

Component	Version	Role
Python	3.13	Runtime
PyTorch	≥ 2.10	Neural network training
Ray / RLlib	≥ 2.54	Distributed RL orchestration
poke-env	≥ 0.12	Gymnasium interface to Showdown
Pokémon Showdown	self-hosted	Battle simulator (Node.js)
MLflow	≥ 3.10	Experiment tracking & metrics
Optuna	≥ 4.8	Hyperparameter optimization (TPE)
Gymnasium	$< 1.2.3$	Environment API
Hardware: NVIDIA RTX 5090, 48 parallel environments, 8 Showdown instances (ports 8000–8007)		

- Package management: `uv` with locked dependencies (`uv.lock`)
- All code open-source; all dependencies permissively licensed

Training Run Flow



Observation Space & Information Asymmetry

Per-turn observation:

- Float matrix: 13×164 (global + 6 own + 6 opp)
- Categorical IDs: species, items, abilities
- Action mask: 14 actions (4 moves + 4 gimmick + 6 switches)

Per-Pokémon token (164 dims):

- Presence flags, HP fraction, base stats
- Types (20-dim multi-hot), status, boosts
- Item/ability flags, moves (4×26 features)

Fixed vs. Random Teams

Fixed (Custom Game):

Team preview shows all 6 opponent species.
Types, base stats known for bench.
Moves/abilities revealed through play.

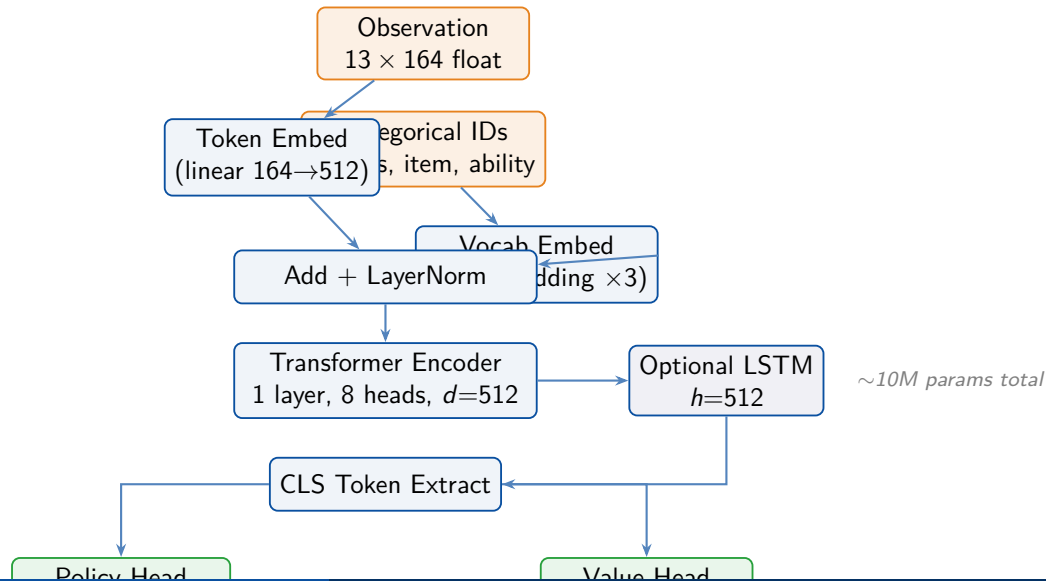
Random (Random Battle):

No team preview. Opponent revealed one-by-one on switch-in. Most of the observation is zeros until mid-battle.

Implication

The M1 vs. M2 performance gap is driven by **information availability**, not just team consistency.

Model Architecture



Reward components (per step):

Signal	Weight / Value
Win bonus	+10
Loss penalty	-10
HP delta	$\times 2.0$
Matchup quality	0.2
Action quality	0.3
Reward scale	$\times 0.1$

Returns scaled from $[-15, +15]$ to $[-1.5, +1.5]$.
Discount: $\gamma = 0.97$, GAE $\lambda = 0.88$.

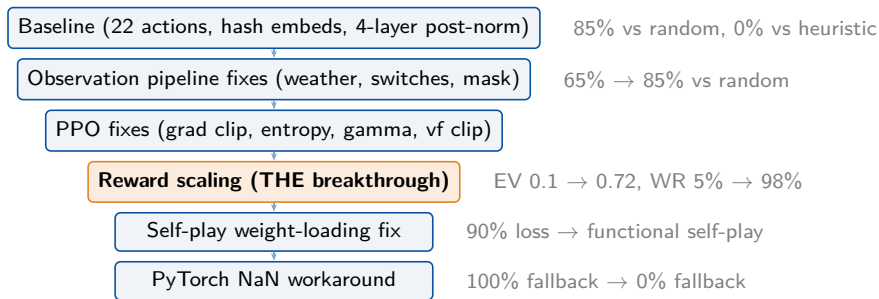
Why reward scale = 0.1?

Without scaling, returns in $[-15, +15]$ made the value function unable to learn (explained variance ~ 0.1). Scaling to $[-1.5, +1.5]$ unlocked value learning: EV jumped to **0.72**. The single most impactful change in the project.

Curriculum stages:

- 1 Random warmup (70% WR threshold)
- 2 Self-play (85% WR, 80/20 mix)
- 3 Mixed terminal (50/30/20 heuristic/self/random)

Development Timeline



Key takeaway: Most “progress” came from fixing bugs and pipeline issues, not from architectural innovation. Systematic debugging was the primary driver.

Critical Bug #1: Gradient Starvation

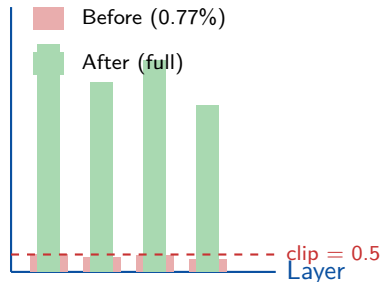
The problem:

- `grad_clip` was set to **0.5**
- Total gradient norm: ~ 65
- After clipping: all gradients scaled to **0.77%** of original
- Transformer and embedding layers received near-zero updates
- Loss metrics looked *normal* – the failure was invisible

The fix: `grad_clip` = 0.5 $\rightarrow$ 5.0

Lesson: Monitor gradient norms, not just losses.

Grad magnitude



Critical Bug #2: Reward Scaling (The Breakthrough)

Metric	Before Scaling	After Scaling
Returns range	$[-15, +15]$	$[-1.5, +1.5]$
Explained variance	~ 0.1 (sometimes neg.)	0.53–0.87 (median 0.72)
Value loss	High, no trend	0.08–0.13 (converged by 15K steps)
WR vs. heuristic	$\sim 5\%$	98% (fixed teams)

The Insight

For the entire project history, the value function was non-functional. A single change, `reward_scale = 0.1`, caused explained variance to jump from ~ 0.1 to 0.72 and unlocked competency against heuristic opponents.

Normalise the return scale before tuning anything else. The value function must be able to learn before the policy can improve.

Critical Bug #3: Self-Play Weight Loading + NaN

Weight loading failure:

- `selfplay_weights_path` passed as *relative* path
- Ray workers run from `/tmp/ray/...`
- Opponent played with **random init** weights
- **0 out of 12** workers loaded weights
- Hidden by silent except: `pass`

Fix: `os.path.abspath()` before passing to workers. Added diagnostic logging to detect load failures.

Lesson: file paths in distributed systems must be absolute.

PyTorch 2.10 NaN bug:

- `TransformerEncoderLayer.forward()` with float `src_mask` → NaN outputs
- Our learnable attention bias triggered it every forward pass
- NaN logits → multinomial error → fallback to random actions
- **100% fallback rate**

Fix: Manual norm-first decomposition replacing the built-in `layer(x, src_mask=bias)`.

Lesson: framework bugs are real. Trusting internals without testing wastes weeks.

Milestone 1: Fixed Teams

Metric	Value
Skill score	0.9911
WR vs. random	100%
WR vs. no-switch	96%+
WR vs. heuristic	98%

Conclusion

The architecture, observation pipeline, and reward design are **sufficient** when team information is available and teams are consistent.

Why does this work so well?

- **Known own team:** The agent learns one team's strategy deeply (memorises matchups, switching patterns).
- **Team preview:** All 6 opponent species visible from turn 1 → can plan switches proactively.
- **Low entropy:** Same team every battle eliminates the need for generalisation across team compositions.

Hparam sweep confirms:

Most configurations reach 96–98% vs. heuristic. Performance is *architecture-agnostic* in this setting.

Milestone 2: Random Teams & The Entropy Problem

Metric	Best Value
Skill score (best trial)	~ 0.37
WR vs. heuristic	$\sim 30\%$

Hparam sweep: 50 trials, 600K steps each

- Fixed teams: saturated at 96–98% across most configs
- Random teams: **no configuration broke through**
- Problem is not opponent AI strength; it's team entropy

The real insight

Our curriculum (opponent strength ramping: random $\rightarrow$ heuristic $\rightarrow$ self-play) turned out to be **less relevant** than initially thought.

The dominant difficulty is **high entropy + partial observability** in random vs. random battles:

- No team preview (opponent revealed one-by-one)
- Every battle is a novel team matchup
- Qualitatively different: POMDP vs. near-MDP

What works	What still needs work
Fixed teams: near-perfect (skill 0.99)	Random teams: skill 0.37, $\sim 30\%$ vs. heuristic
Value function learns after reward scaling (EV 0.72)	Benchmark variance high at 50 eps/tier
Self-play functional (0% fallback)	LSTM end-to-end validation pending
Distributed training stable at 48 envs	Gauntlet validation not automated
MLflow produces complete, comparable metrics	Team builder (M3) not started
Curriculum learning provides structured progression	Partial observability remains the core challenge

Bottom line: The engineering foundation is solid and M1 is solved. The remaining challenge is fundamental: generalisation under hidden information with high team entropy.

Live Demo

Agent vs. Heuristic Opponent

Fixed team, gen8customgamenogimmicks format

Lessons Learned

- 1 **Normalise the return scale first.** Reward scaling ($\times 0.1$) was more impactful than any architectural change. The value function must learn before the policy can improve.
- 2 **Systematic debugging > throwing compute at the problem.** Three observation bugs collectively affected 14% of steps. A silent weight-loading bug wasted weeks of self-play training.
- 3 **Reward design outweighs architecture tuning.** The biggest gains came from fixing observation bugs (WR 65% $\rightarrow$ 85%) and reward scaling (WR 5% $\rightarrow$ 98%), not from model changes.
- 4 **Curriculum design must target the right axis of difficulty.** Opponent strength was less important than information availability. High entropy in team composition is the real bottleneck.
- 5 **Framework bugs happen.** PyTorch 2.10 NaN with float attention masks required manual decomposition. Trusting framework internals wastes time.

Next Steps & Future Work

Short term (continuing the project):

- Gradually expand from fixed teams at win-rate thresholds (bridge M1 $\rightarrow$ M2)
- Complete-information random vs. random as controlled experiment
- Increase benchmark stability (100+ eps/tier)
- Validate LSTM benefit in A/B test

Longer term:

- **Pretraining on Showdown logs** + RL finetuning: likely the highest-impact change not attempted
- Auxiliary tasks (move prediction, state-value supervision)
- Milestone 3: team composition model
- Milestone 4: full Nuzlocke gauntlet

Project Outlook

The project continues beyond the DSPRO2 module. The engineering foundation (distributed training, validation, experiment tracking) is solid. The remaining challenge: generalisation under partial observability.

Questions?

Thank you for your attention

Tristan Muri, Marvin Hueber, Viacheslav Godovskii, Alex Liubov

DSPRO2 FS26 – Hochschule Luzern